

sigc 관찰 프로브

실신호 특징 4줄 + find_bursts 단계 관찰 — [(0, n)] 의 이유를 현장에서 본다

코드 204줄 · 총 5쪽 | 2026-08-14 · 92f4c81+수정중 | 의존성: 설치돼 있는 signus + numpy/scipy (matplotlib 이 있고 창을 띄울 수 있으면 그림 — ssh/무화면이면 자동으로 ASCII)

- 이 코드를 **signus/** 폴더 **옆에 sigc.py** 로 저장한다. 필사는 두 토막으로 끊어도 된다 — 「여기부터는 그림 단계」 주석 앞까지만 옮기면 ②③(요약 4줄) 이 다 돌고, 그 뒤는 ④의 그림 단계 전용이다.
- python3 sigc.py kat** — 아래 기대 4줄과 글자까지 비교. 다르면 스크립트 필사 오타다 (kat 이 잡아준다).
- python3 sigc.py <캡처파일>** — 요약 4줄을 #코드까지 그대로 받아쳐 회신한다. (파일명은 analyze 때처럼 fs·iq/real 을 실어야 한다)
- python3 sigc.py <캡처파일> 2** 처럼 단계 번호를 붙이면 그 단계를 그림/ASCII 로 보여준다 — 본 것을 말로 회신하면 된다.
- 받은 쪽(맥)은 **tools/sigc.py check** 로 먼저 오타 검증 + 출구 해석을 한다.

```
sigc a n66000 f1000000 ev86 ed55 sp26 dc0 cp4 cx1 #5zx2
sigc b c1030 g128 b155 cn156 t308 m520 dlo25 dhi45 #7hny
sigc c r7 dn97 gp90 sb33 av66 ah56 sk1 #vaw0
sigc d kb17 kc8 w7 p-14 pd56 ps59 q37 qd100 qs51 #7m4q
```

명령 인자	무엇을 보나	kat 에서 보여야 하는 것	실캡처에서 관찰·회신할 것
kat	자기검증	아래 기대 4줄과 글자까지 일치	매번 먼저. 다르면 값이 다른 줄부터 스크립트 재대조
(없음)	요약 4줄	kat 과 같은 형식	4줄을 #코드까지 그대로 받아쳐 회신
1	읽기 확인	n 66000 · 길이 0.066s · 형식 iq	n·fs 가 캡처 실제와 맞는가
2	광대역 포락선 (베타)	계단이 또렷하고 분리도 0.86	계단(#)과 골(빈칸)이 보이는가, 분리도가 0.12 를 넘는가
3	위터폴 절대전력	세로 버스트 줄 + 가로로 안 끊기는 띠 2개	끊이지 않는 가로 띠(연속 방사체)가 있는가, 버스트 줄 수
4	빈별 바닥 대비 비율	세기 일정한 연속 띠는 사라지고 버스트만	find_bursts 가 보는 그림 — 버스트가 여기서도 살아 있는가
5	열 점수 + 문턱	봉우리가 hi 위, base 1.55 ≈ c_noise 1.56	봉우리가 hi 선을 넘는가, base 가 c_noise 근처인가
6	후보 런 + 실제 답	원시 후보 7개 → find_bursts → [(64, 6080), (11904, 180	원시 후보가 병합·가드를 거쳐 몇 개로 남는가

요약 4줄 읽는 법: a=포락선(ev 분리도x100 · ed 듀티% · sp 피크비dB · dc% · cp 포화 지표%) = 캡처 자신의 상위 0.1% 레벨에 붙은 표본 비율, 깨끗하면 2~4·포화면 900+ · cx1=복소/cx0=실수) b=셀 점수 지형(c 열수 · g 빈수 · b base · cn · t 문턱 · m 최대 · dlo/dhi 는 base 로부터의 문턱 여유, x100 — dlo25 dhi45 가 아니면 그 상수를 잘못 옮긴 것) c=버스트 구조(r 런수 · dn 길이열 · gp 간격열 · sb 점수dB · av/ah 문턱 위 열% · sk 스파이크런) d=방사체(kb 점유빈 · kc 연속점유빈 · w 최대폭빈 · p/q 위치빈 · pd/qd 듀티% · ps/qs 절대dB, q999=부 방사체 없음). 한 줄이 100칸을 넘으면 ↵ 이어짐 — 붙여서 입력. **kat 이 검증하는 것은 요약 4줄 경로 뿐이다** — 단계 2~6 의 그림 코드(curve/image/pool)는 kat 이 실행하지 않으므로, 그림 단계에서 처음 크래시하면 그 세 함수의 필사부터 다시 본다.

```

1  """sigc.py - 실신호 특징 4줄 + find_bursts 단계 관찰 프로브 (장비 필사용, signus/ 옆에 저장).
2      python3 sigc.py kat                # 자기검증 - 표지의 기대 4줄과 비교
3      python3 sigc.py <캡처> [1-6]       # 요약 4줄(받아쳐 회신) / 단계 관찰(그림 또는 ASCII)
4  """
5  import sys
6
7  import numpy as np
8  from scipy.ndimage import uniform_filter1d
9  from scipy.signal import get_window, stft
10
11 from signus import dsp, sigio
12 from signus.cli import check_code
13
14
15 def otsu(v, bins=128):
16     h, ed = np.histogram(v, bins=bins)
17     c = (ed[:-1] + ed[1:]) / 2
18     w = np.cumsum(h).astype(float)
19     m = np.cumsum(h * c)
20     mb = m / np.where(w > 0, w, 1)
21     mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
22     return float(c[int(np.argmax(w * (w[-1] - w) * (mb - mf) ** 2))])
23
24
25 def runs_of(mask):
26     d = np.diff(mask.astype(np.int8))
27     s = list(np.where(d == 1)[0] + 1)
28     e = list(np.where(d == -1)[0] + 1)
29     if mask[0]:
30         s.insert(0, 0)
31     if mask[-1]:
32         e.append(mask.size)
33     return list(zip(s, e, strict=True))
34
35
36 if len(sys.argv) < 2:
37     raise SystemExit("사용법: python3 sigc.py kat | python3 sigc.py <캡처파일> [단계 1-6]")
38 arg, step = sys.argv[1], int(sys.argv[2]) if len(sys.argv) > 2 else 0
39 if arg == "kat":
40     fs, t = 1e6, np.arange(66000.0)      # 66000: 마지막 버스트가 파일 끝까지 이어진다
41     rng = np.random.default_rng(7)       # 시드 고정 스트림 - numpy 가 재현을 보장한다
42     x0 = 0.01 * (rng.standard_normal(66000) + 1j * rng.standard_normal(66000))
43     x0 += 0.2 * np.exp(2j * np.pi * 0.29 * t)
44     x0 += 0.5 * np.exp(-2j * np.pi * 0.11 * t) * ((t // 6000).astype(int) % 2 == 0)
45     x0 += 0.0028 * np.exp(2j * np.pi * 0.41 * t)    # 문턱 경계에 걸친 성분들: 필사 오타가
46     x0[31000:35000] += 0.0088 * np.exp(2j * np.pi * 0.35 * t[:4000])    # 상수 하나만
47     x0[42400:47600] += 0.0075 * np.exp(2j * np.pi * -0.37 * t[:5200])    # 스쳐도 아래
48     x0[200:260] = 0.95                # 요약 4줄 어딘가가 바뀌게 만든다
49     x0[21000] += 8.0
50     x0[15000] += 2.0
51 else:
52     x0, meta = sigio.read(arg)
53     fs = meta.fs
54 cx = np.iscomplexobj(x0)
55 raw = np.abs(np.asarray([x0.real, x0.imag])).max(0) if cx else np.abs(x0)
56 cp = float((raw > 0.98 * np.percentile(raw, 99.9)).mean())    # 만점(±1)이 아니라 캡처 자신의
57 # 99.9퍼센타일 기준: float 캡처는 만점 정규화가 안 돼 ±1 기준이면 통짜 오경보다(실측 cp972)
58 x = dsp.analytic(x0)
59 n, pw = x.size, np.abs(x) ** 2
60 rms = float(np.sqrt(pw.mean()) + 1e-30)

```

```

61 dc = abs(complex(x.mean())) / rms          # dc 는 블록 전에 재고, 그 뒤는 pipeline 과 똑같이
62 x = x - x.mean()                          # DC 블록한 신호로 켜다 - 안 그러면 dc 큰 캡처에서
63 pw = np.abs(x) ** 2                       # 프로브와 analyze 가 정반대 진단을 낸다
64 # 러닝합 잔차 때문에 정확히 0 인 구간(스켈치/뮤트)에서 음수가 나와 log10 이 NaN 이 되고,
65 # otsu 의 histogram 이 장비에서 생 트레이스백으로 죽었다 - 0 으로 눌러 막는다.
66 lp = np.log10(np.maximum(uniform_filter1d(pw, max(64, n // 1000)), 0) + 1e-20)
67 hot = lp >= (et := otsu(lp))
68 ev = float(lp[hot].mean() - lp[~hot].mean()) if 0 < hot.sum() < n else 0.0
69 ed = float(hot.mean())
70 sp = 10 * np.log10(pw.max() / (pw.mean() + 1e-30) + 1e-30)
71 nper = int(min(128, max(64, 1 << int(np.log2(max(n // 2, 64)))))
72 hop = nper // 2
73 _, _, z = stft(x, fs=fs, window=get_window("blackmanharris", nper), nperseg=nper,
74               noverlap=nper - hop, return_onesided=False, boundary=None, padded=False)
75 P = np.abs(z) ** 2
76 nb, nc = P.shape
77 fl = np.percentile(P, 10, axis=1)
78 r = P / (fl[:, None] + 1e-30)
79 k = max(3, nb // 16)
80 sc = uniform_filter1d(np.log10(np.sort(r, axis=0)[-k:].mean(axis=0) + 1e-30), 3)
81 thr = otsu(sc, bins=64)
82 quiet = sc[sc < thr]
83 base = float(np.median(quiet)) if quiet.size else 9.99
84 cn = float(np.log10((np.log(nb / k) + 1) / 0.105))
85 lov, hiv = base + 0.25, base + 0.45      # find_bursts 의 두 문턱. 각 상수는 한 곳에만 쓰고,
86 above, hi_m = sc >= lov, sc >= hiv      # b 줄에 dlo/dhi 로 되찍는다 - 오타면 그 숫자가 바뀐다
87 rr = [(s, e) for s, e in runs_of(above) if hi_m[s:e].any()]
88 spans = [(max(0, s * hop) + (hop if e - s >= 6 else 0),          # 6월 이상은 양끝 한 hop
89           min(n, (e - 1) * hop + nper) - (hop if e - s >= 6 else 0)) # 썩 자른다 - dsp 의
90           for s, e in rr]                                         # 스파이크 게이트와 같은
91 segs = [pw[s:e] for s, e in spans]                                # 구간에서 재려고
92 sk = sum(float(sg.max() / (sg.mean() + 1e-30)) > 60.0 for sg in segs)
93 gp_ = np.array([rr[i + 1][0] - rr[i][1] for i in range(len(rr) - 1)])
94 sb = round(10 * float(np.median([sc[s:e].max() for s, e in rr]) - base)) if rr else 0
95 g = float(np.median(fl if cx else fl[:nb // 2]) + 1e-30)      # real 은 해석신호라 음수 반쪽이
96 # 비어 있다: 전 빈 중앙값을 쓰면 바닥이 0 으로 내려가 밴드 절반이 '점유'로 읽힌다(실측)
97 Ps, fls = np.fft.fftshift(P, 0), np.fft.fftshift(fl)
98 du = (Ps > 40 * g).mean(1)
99 occ = du > 0.1
100 kb, kcont = int(occ.sum()), int((fls > 5 * g).sum())
101 grp = runs_of(occ) if occ.any() else []
102 wd = max((e - s for s, e in grp), default=0)
103 p95 = np.percentile(Ps, 95, axis=1) / g
104 pk = int(np.argmax(p95))
105 pgrp = next(((s, e) for s, e in grp if s <= pk < e), (pk, pk + 1))
106 m2 = np.where(occ, p95, 0.0)
107 m2[pgrp[0]:pgrp[1]] = 0.0
108 q2 = int(np.argmax(m2))
109 qo, qd, qs = (q2 - nb // 2, round(100 * float(du[q2])),
110               round(10 * np.log10(m2[q2] + 1e-30))) if m2[q2] > 0 else (999, 0, 0)
111 la = (f"sigc a n{n} f{fs:.0f} ev{round(100 * ev)} ed{round(100 * ed)}"
112       f" sp{round(float(sp))} dc{round(100 * dc)} cp{round(1000 * cp)}"
113       f" cx{int(cx)}")
114 lb = (f"sigc b c{nc} g{nb} b{round(100 * base)} cn{round(100 * cn)}"
115       f" t{round(100 * thr)} m{round(100 * float(sc.max()))}"
116       f" dlo{round(100 * (lov - base))} dhi{round(100 * (hiv - base))}")
117 lc = (f"sigc c r{len(rr)} dn{round(float(np.median([e - s for s, e in rr])))} if rr else 0}"
118       f" gp{round(float(np.median(gp_))} if gp_.size else 0} sb{sb}"
119       f" av{round(100 * float(above.mean()))} ah{round(100 * float(hi_m.mean()))}"
120       f" sk{sk}")

```

```

121 ld = (f"sigc d kb{kb} kc{kcont} w{wd} p{pk - nb // 2} pd{round(100 * float(du[pk]))}"
122      f" ps{round(10 * np.log10(p95[pk] + 1e-30))} q{qo} qd{qd} qs{qs}")
123 if step == 0:                                # 요약 4줄 - 이것만 받아쳐서 회신한다
124     for s in (la, lb, lc, ld):
125         print(f"{s} #{check_code(s)}")
126     sys.exit()
127
128 # — 여기부터는 그림 단계(1~6) 전용이다. 요약 4줄만 쓸 거면 여기서 멈춰도 된다 —
129
130 try:
131     import matplotlib.pyplot as plt
132     plt = None if plt.get_backend().lower() == "agg" else plt    # 무화면(ssh)이면 ASCII 로
133 except ImportError:
134     plt = None
135
136
137 def pool(a, m):
138     a = np.asarray(a, float)
139     k = max(1, a.shape[-1] // m)
140     return a[..., :(a.shape[-1] // k) * k].reshape(*a.shape[:-1], -1, k).max(-1)
141
142
143 def curve(y, marks):
144     if plt:
145         plt.plot(y)
146         for la, v in marks:
147             plt.axhline(v, ls="--", label=f"{la} {v:.2f}")
148         plt.legend()
149         return plt.show()
150     # 얼마나 최소·최대를 같이 그린다: max 만 그리면 버스트 사이의 골이 지워져 계단이 통짜
151     # 블록으로 보이고, 관측자가 "평평하다(=베토)" 라고 정반대로 회신하게 된다.
152     kk = max(1, y.size // 100)
153     y2 = y[:y.size // kk * kk].reshape(-1, kk)
154     ylo, yhi = y2.min(1), y2.max(1)
155     bot = float(min(ylo.min(), *(v for _, v in marks)))
156     st = (float(max(yhi.max(), *(v for _, v in marks))) - bot) / 18 or 1.0
157     rl, rh = np.round((ylo - bot) / st).astype(int), np.round((yhi - bot) / st).astype(int)
158     rm = {}
159     for la, v in marks:
160         rm.setdefault(round((v - bot) / st), []).append(f"{la} {v:.2f}")
161     for r in range(18, -1, -1):
162         bg = "-" if r in rm else " "                # 문턱은 가로 전체 선으로 (겹치면 라벨 합침)
163         print("".join("#" if rl[i] >= r else ("+" if rh[i] >= r else bg)
164               for i in range(rl.size)) + "|" + " / ".join(rm.get(r, [])))
165     print("#=이 구간 내내 그 위 · +=봉우리만 그 위 · -=문턱선")
166
167
168 def image(img):
169     if plt:
170         plt.imshow(img, aspect="auto", origin="lower", interpolation="nearest")
171         return plt.show()
172     sm = pool(pool(img, 100).T, 32).T
173     hi = sm.max() + 1e-30
174     lo = max(float(np.percentile(sm, 5)), hi - 4)    # 표시폭 4 decades 고정 - real 캡처는 빈
175     # 음수 반쪽이 1e-16 이하 5퍼센타일을 쓰면 눈금이 14 decades 로 늘어나 통짜 벽이 된다
176     for row in np.clip((sm - lo) / (hi - lo) * 9.99, 0, 9).astype(int)[::-1]:
177         print("".join(" .:~+*#%@"[i] for i in row))
178
179
180 if step == 1:

```

```

181     print(f"\n {n} fs {fs:.0f} 길이 {n / fs:.3f}s 형식 {'iq' if np.iscomplexobj(x0) else 'real'}")
182     print(f"rms {rms:.4f} dc {100 * dc:.1f}% 클리핑 {1000 * cp:.1f}% 피크비 {float(sp):.0f}dB")
183     elif step == 2:
184         curve(lp, [("otsu", et)])
185         print(f"포락선 분리도 {ev:.3f} decades - 0.12 미만이면 find_bursts 는 통짜 [(0,n)] (베토)")
186         print(f"포락선 듀티 {100 * ed:.0f}% (버스트가 계단으로 보여야 정상)")
187     elif step == 3:
188         image(np.log10(Ps + 1e-20))
189         print("위터폴 절대전력 (아래=-fs/2, 위=+fs/2) - 가로로 끊기지 않는 띠 = 연속 방사체")
190     elif step == 4:
191         image(np.log10(np.fft.fftshift(r, 0) + 1e-30))
192         print("빈별 바닥 대비 비율 - find_bursts 가 보는 그림. 세기가 일정한 연속 방사체는 여기서"
193               " 사라지지만, 세기가 흔들리면 가짜 버스트로 남는다 (d 줄 pd 로 확인)")
194     elif step == 5:
195         curve(sc, [("base", base), ("hi", hiv), ("c_noise", cn)])
196         print(f"열 점수 - hi 위 봉우리가 버스트 후보. 런 {len(rr)}개, base {base:.2f}, cn {cn:.2f}")
197     elif step == 6:
198         for (s, e), (a0, a1), sg in zip(rr, spans, segs, strict=True):
199             print(f"열 {s}-{e} 샘플 {a0}-{a1} 높이 {10 * (float(sc[s:e].max()) - base):.0f}dB"
200                   f" 피크/평균 {float(sg.max() / (sg.mean() + 1e-30)):.0f} (60 초과=스파이크 탈락)")
201         print(f"위는 병합/가드 전 원시 후보 {len(rr)}개 - 아래가 실제로 분석에 쓰이는 답이다:")
202         fb = dsp.find_bursts(x, fs)
203         print(f"find_bursts → {fb[:6]}{'...' if len(fb) > 6 else ''}"
204               + (" ※ 통짜 = 버스트 미검출" if fb == [(0, n)] else f" 버스트 {len(fb)}개"))

```

참고편 — 단계별로 무엇이 보여야 하는가

아래 그림은 전부 이 프로브를 합성 신호에 실제로 돌린 출력이다. 장비에서 나오는 그림과 같은 코드·같은 조판이므로, 모양을 그대로 견주면 된다. 가로축은 언제나 레코드 전체를 100칸으로 줄인 것이고, 캡처 길이가 달라도 모양은 비교할 수 있다.

곡선 그림(단계 2·5) 읽는 법: 한 칸은 여러 샘플을 묶은 것이라 그 구간의 최솟값과 최댓값을 같이 그린다. #=그 구간이 내내 이 높이 위, +=봉우리만 이 높이 위, 빈칸=아래. 가로로 이어지는 ---- 선은 문턱이고 오른쪽에 이름이 붙는다. 버스트열이면 #기둥과 빈칸이 번갈아 나오는 '계단' 이 보여야 한다.

위터폴 그림(단계 4) 읽는 법: 세로가 주파수(아래 $-fs/2$, 위 $+fs/2$), 가로가 시간이다. 진하기는 그 칸이 자기 주파수의 바닥보다 몇 배 센가를 뜻한다(연한 . 부터 진한 @ 까지). 세로로 짧게 끊긴 진한 자국=버스트, 가로로 안 끊기고 쪽 이어지는 띠=연속 방사체다.

각 시나리오 머리의 '요약 4줄' 은 그 신호를 프로브에 넣었을 때 나오는 회신이다. 실캡처의 4줄과 숫자대를 견주면 어느 시나리오에 가까운지 바로 짚인다.

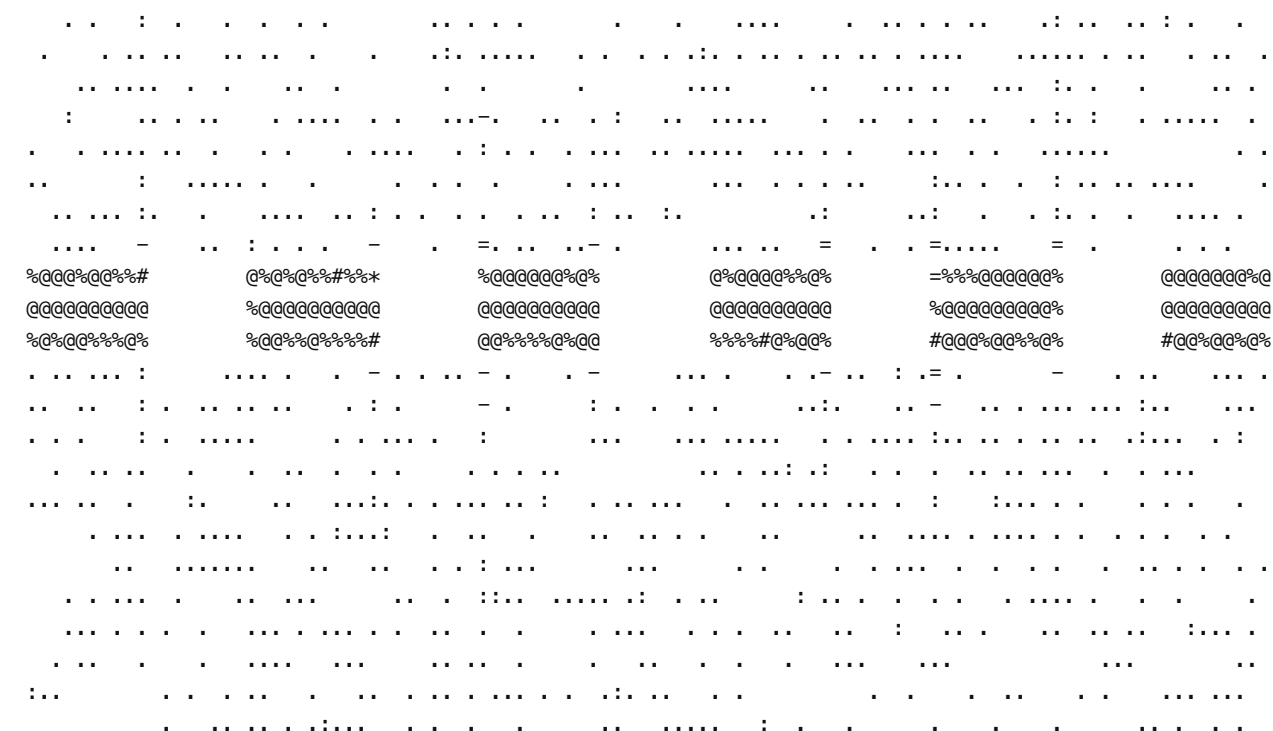
협대역 QPSK 한 개(+200 kHz, 심볼레이트 50 k)가 6000샘플 켜지고 6000샘플 꺼지기를 반복. 잡음 대비 20 dB. 다른 방사체는 없다.

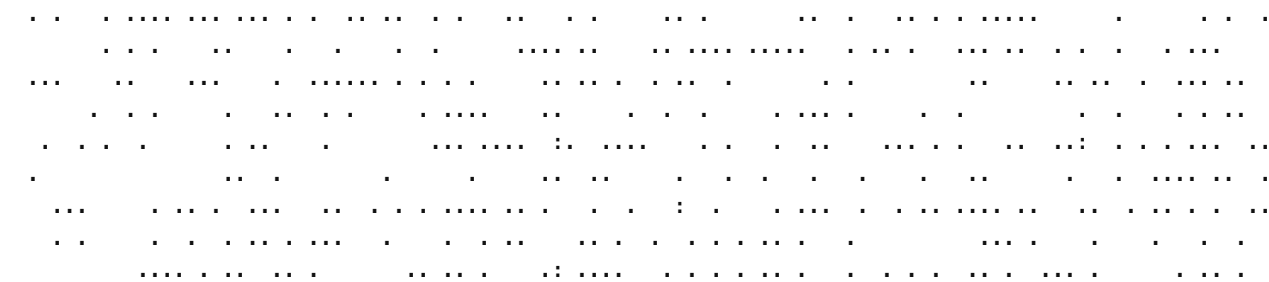
```
sigc a n65536 f1000000 ev200 ed55 sp7 dc0 cp2 cx1 #r064
sigc b c1023 g128 b152 cn156 t263 m385 dlo25 dhi45 #9fzy
sigc c r6 dn96 gp91 sb23 av56 ah55 sk0 #rqgp
sigc d kb12 kc0 w12 p26 pd54 ps46 q999 qd0 qs0 #zgb3
```

단계 2 — 광대역 포락선 (여기서 막히면 E1 베토)



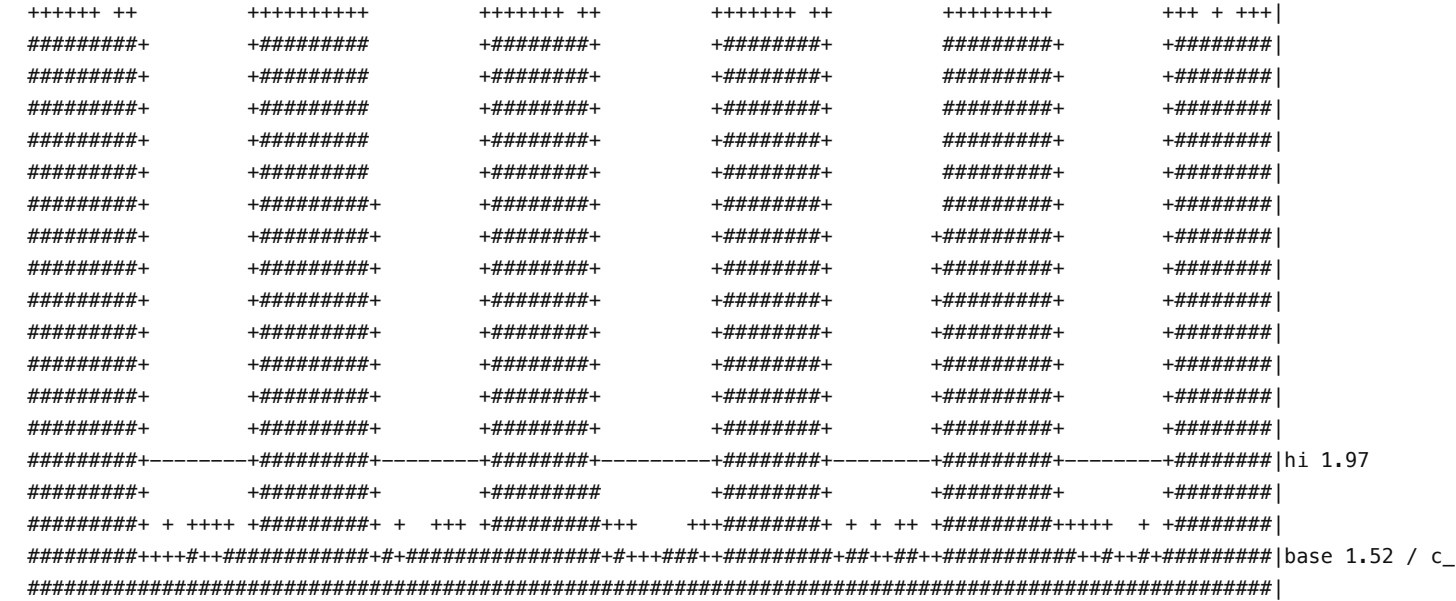
단계 4 — 빈별 바닥 대비 비율 (find_bursts 가 보는 그림)





bin별 바닥 대비 비율 — find_bursts 가 보는 그림. 세기가 일정한 연속 방사체는 여기서 사라지지만, 세기가 흔들리면 가짜 버스트로 남는다 (d 줄 pd 로 확인)

단계 5 — 열 점수와 문턱 (붕우리가 hi 를 넘어야 후보)



#=이 구간 내내 그 위 · +=붕우리만 그 위 · -=문턱선

열 점수 — hi 위 붕우리가 버스트 후보. 런 6개, base 1.52, cn 1.56

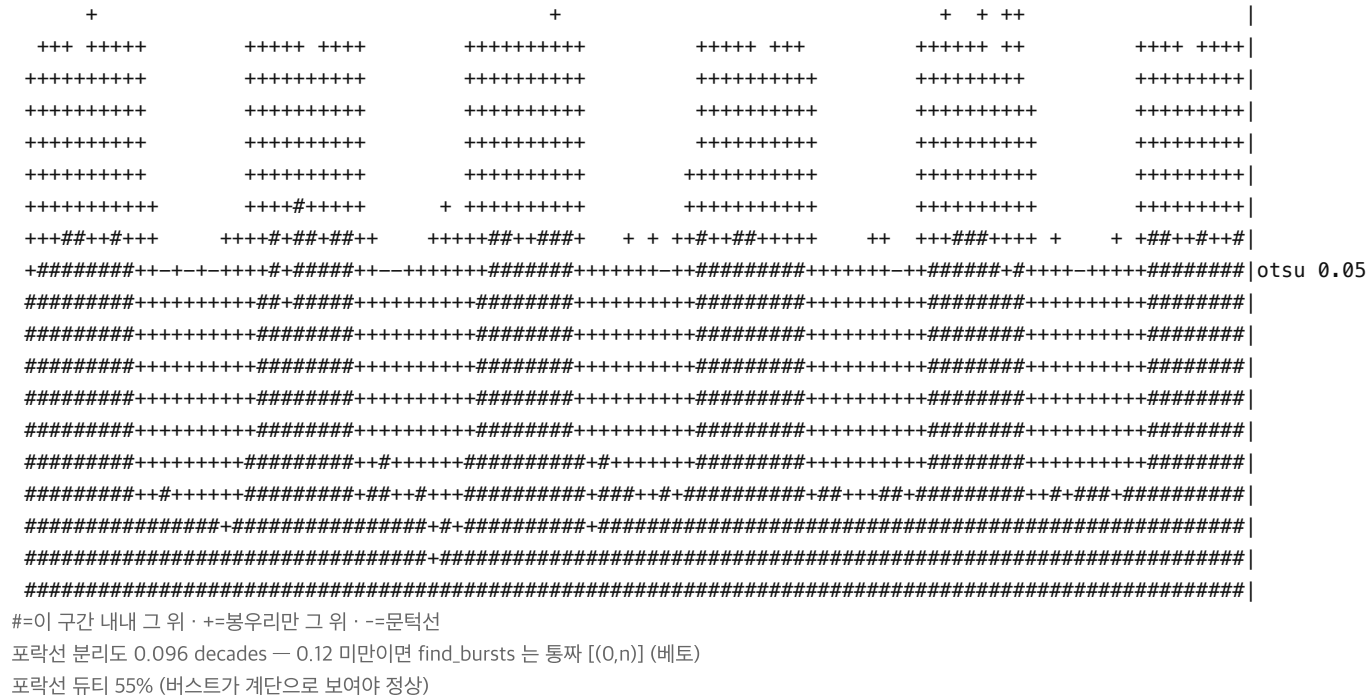
실캡처가 이렇게 보이면

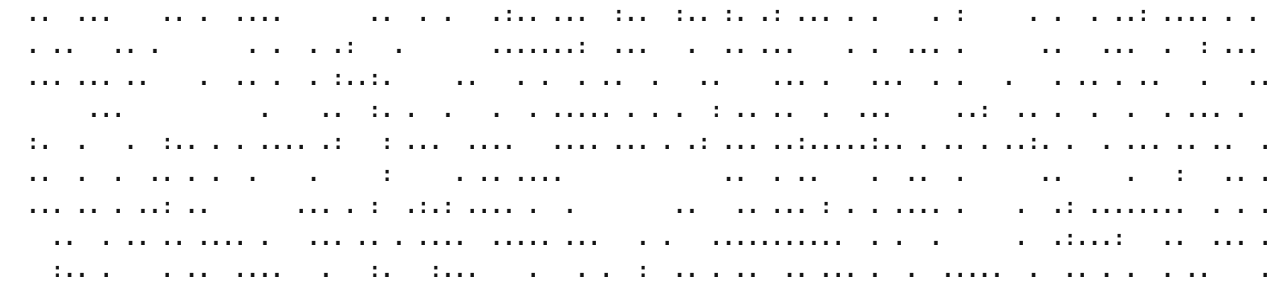
실캡처가 이 모양이면 find_bursts 는 버스트를 잡는다. 그런데도 통짜가 나온다면 4줄이 아니라 단계 6(실제 출력)을 봐야 한다 — 병합이나 1열 드랍이 원인이다.

A 와 같은 버스트열(세기는 절반)에, 레코드 내내 꺼지지 않는 연속 QPSK(-150 kHz, 심볼레이트 80 k)가 함께 들어온다. 합성 시험에 없던 조합이다.

```
sigc a n65536 f1000000 ev10 ed55 sp6 dc0 cp2 cx1 #7tc0
sigc b c1023 g128 b151 cn156 t236 m332 dlo25 dhi45 #e1bt
sigc c r6 dn96 gp91 sb18 av55 ah55 sk0 #fxrs
sigc d kb29 kc16 w17 p-20 pd100 ps45 q25 qd54 qs40 #778s
```

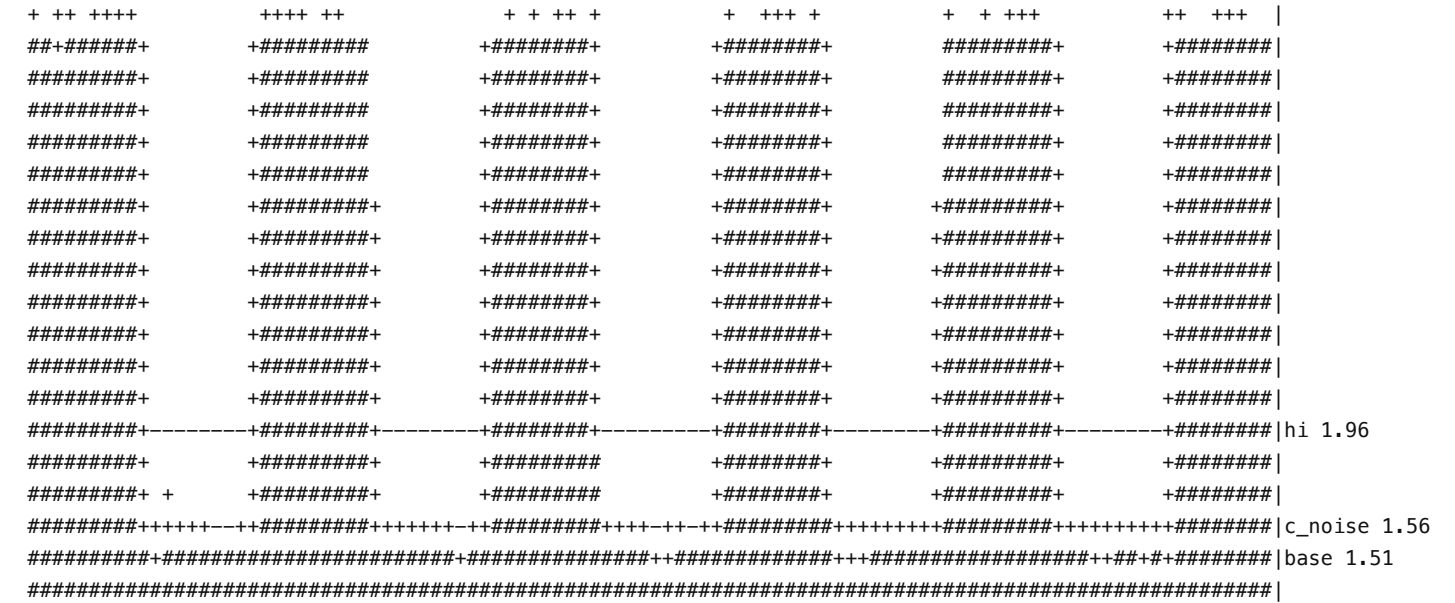
단계 2 — 광대역 포락선 (여기서 막히면 E1 베토)

[illegible]



빈별 바닥 대비 비율 — find_bursts 가 보는 그림. 세기가 일정한 연속 방사체는 여기서 사라지지만, 세기가 흔들리면 가짜 버스트로 남는다 (d 줄 pd 로 확인)

단계 5 — 열 점수와 문턱 (붕우리가 hi 를 넘어야 후보)



#=이 구간 내내 그 위 · +=붕우리만 그 위 · -=문턱선
열 점수 — hi 위 붕우리가 버스트 후보. 런 6개, base 1.51, cn 1.56

실캡처가 이렇게 보이면

d 줄에 pd100(듀티 100%)인 방사체가 있고 kc 가 0 보다 크면 이 경우다. 버스트는 단계 4·5 에 멀쩡히 살아 있는데, 광대역 포락선만 평평해져 셀 경로에 닿기도 전에 막힌다.

A 와 같은 신호인데 12000샘플 커지고 600샘플만 꺼진다. 빈별 바닥이 신호를 '늘 있는 것' 으로 학습해 버려 점수가 서지 않는다.

```
sigc a n65536 f1000000 ev199 ed96 sp5 dc0 cp2 cx1 #sn8v
sigc b c1023 g128 b154 cn156 t157 m191 dlo25 dhi45 #4jbk
sigc c r0 dn0 gp0 sb0 av1 ah0 sk0 #mdak
sigc d kb12 kc12 w12 p25 pd95 ps47 q999 qd0 qs0 #3c55
```

단계 2 — 광대역 포락선 (여기서 막히면 E1 베토)



#=이 구간 내내 그 위, +=붕우리만 그 위, -=문턱선
 포락선 분리도 1.992 decades — 0.12 미만이면 find_bursts 는 통짜 [(0,n)] (베토)
 포락선 듀티 96% (버스트가 계단으로 보여야 정상)

[illegible]

#=이 구간 내내 그 위 · +=봉우리만 그 위 · -=문턱선
열 점수 — hi 위 봉우리가 버스트 후보. 런 0개, base 1.54, cn 1.56

실캡처가 이렇게 보이면

a 줄 ed 가 90 이상이고 c 줄 r0(런 없음)이면 이 경우다. 듀티 90% 까지는 잡히고 95% 부터 무너지는 것이 실측이다 — 설계 한계이지 오작동이 아니다.

D. 약한 협대역 — 잡히긴 하는데 여유가 6 dB (경계)

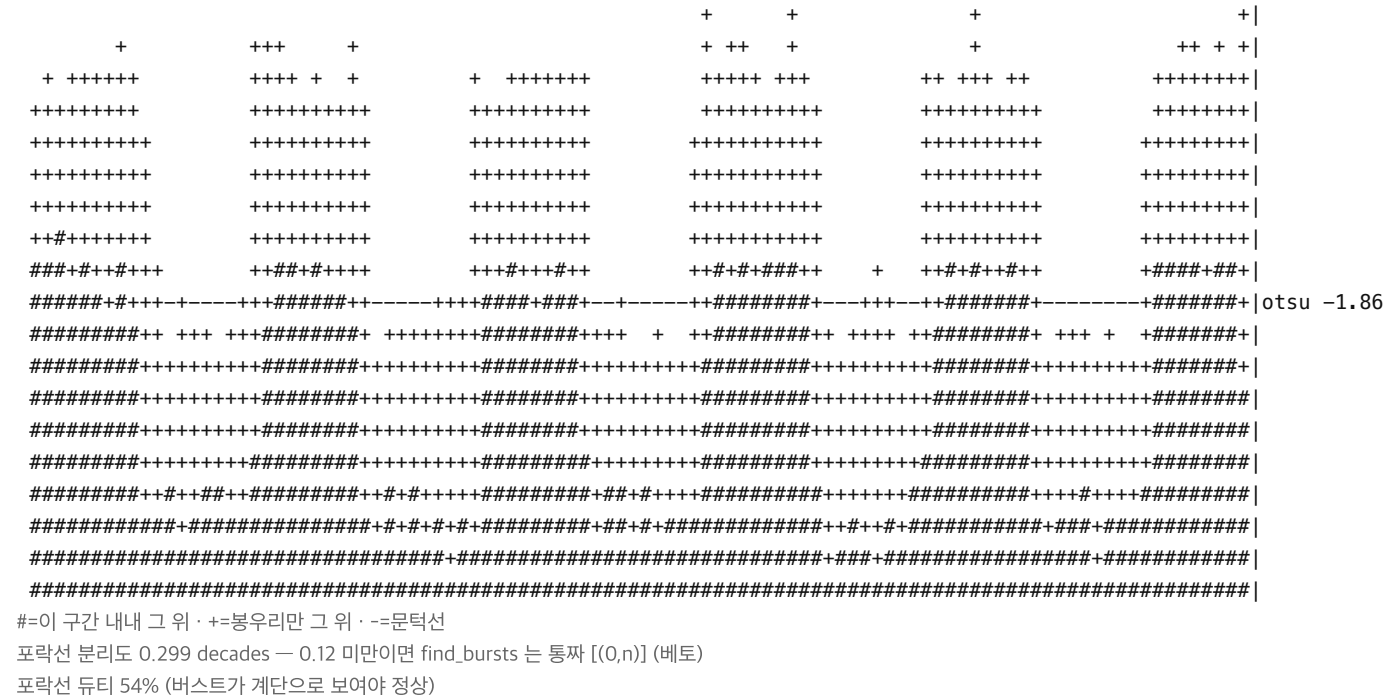
5빈짜리 아주 좁은 QPSK(+100 kHz, 심볼레이트 20 k)가 잡음에 겨우 묻히지 않을 세기로 들어온다. 커짐/꺼짐 주기는 A와 같다.

이 신호를 프로브에 넣으면 나오는 4줄

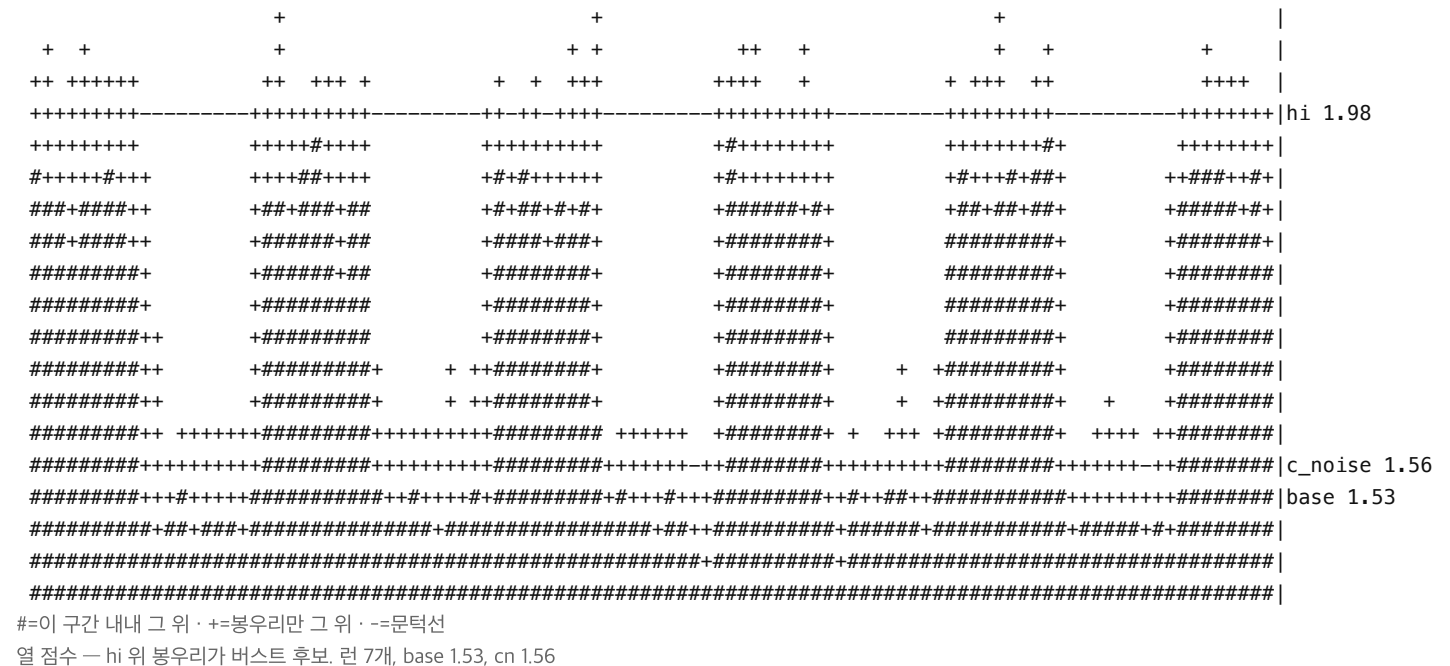
```
sigc a n65536 f1000000 ev30 ed54 sp10 dc0 cp1 cx1 #ednt
sigc b c1023 g128 b153 cn156 t172 m211 dlo25 dhi45 #wmwq
sigc c r7 dn93 gp94 sb6 av54 ah17 sk0 #shpf
sigc d kb5 kc0 w5 p13 pd53 ps29 q999 qd0 qs0 #tbg5
```

find_bursts → 버스트 6개 [(64, 5952), (12096, 17920), (24064, 29952)]...

단계 2 — 광대역 포락선 (여기서 막히면 E1 베토)



단계 5 — 열 점수와 문턱 (붕우리가 hi 를 넘어야 후보)



실캡처가 이렇게 보이면

c 줄 sb 가 6 안팎이면 경계다(건강한 A 는 23). 여기서 몇 dB 만 더 낮아지면 통째로 넘어간다. 잡히긴 해도 이 여유로는 신뢰하지 말고 단계 5 를 같이 볼 것.

E. 임펄스 간섭 — 모든 버스트에 스파이크 (E6)

A 와 같은 버스트열의 버스트마다 한 샘플짜리 강한 임펄스(+30)가 박혀 있다. 전원 잡음이나 스위칭 잡음이 이렇게 보인다.

이 신호를 프로브에 넣으면 나오는 4줄

```
sigc a n65536 f1000000 ev201 ed55 sp32 dc0 cp1 cx1 #vz3g
sigc b c1023 g128 b151 cn156 t267 m404 dlo25 dhi45 #njsx
sigc c r6 dn96 gp91 sb25 av55 ah55 sk6 #556j
sigc d kb12 kc0 w12 p26 pd54 ps46 q999 qd0 qs0 #zgb3
```

find_bursts → 통짜 [(0, 65536)] = 버스트 미검출

단계 2 — 광대역 포락선 (여기서 막히면 E1 베토)

```

+           +           +           +           +           +           |
+           +           +           +           +           +           |
+           +           +           +           +           +           |
+           +           +           +           +           +           |
+           +           +           +           +           +           |
+           +           +           +           +           +           |
+++++++      ++++++      ++++++      ++++++      ++++++      ++++++ |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+-----+#####+-----+#####+-----+#####+-----+##### |otsu -1.01
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+++++++#####+++++++#####+++++++#####+++++++#####+++++++ |
#####+#####+#####+#####+#####+#####+#####+#####+#####+##### |
##### |
#=#이 구간 내내 그 위 · +=붕우리만 그 위 · -=문턱선
포락선 분리도 2.015 decades — 0.12 미만이면 find_bursts 는 통짜 [(0,n)] (베토)
포락선 듀티 55% (버스트가 계단으로 보여야 정상)
```

단계 5 — 열 점수와 문턱 (붕우리가 hi 를 넘어야 후보)

```

+           +           +           +           +           +           |
++++++ +++++      ++++++      ++++++      ++++++      ++++++      ++++++ |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+-----+#####+-----+#####+-----+#####+-----+##### |hi 1.96
#####+      +#####+      +#####+      +#####+      +#####+      +##### |
#####+ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + + |
#####+#####+#####+#####+#####+#####+#####+#####+#####+##### |base 1.51 / c_
##### |
#=#이 구간 내내 그 위 · +=붕우리만 그 위 · -=문턱선
열 점수 — hi 위 붕우리가 버스트 후보. 런 6개, base 1.51, cn 1.56
```

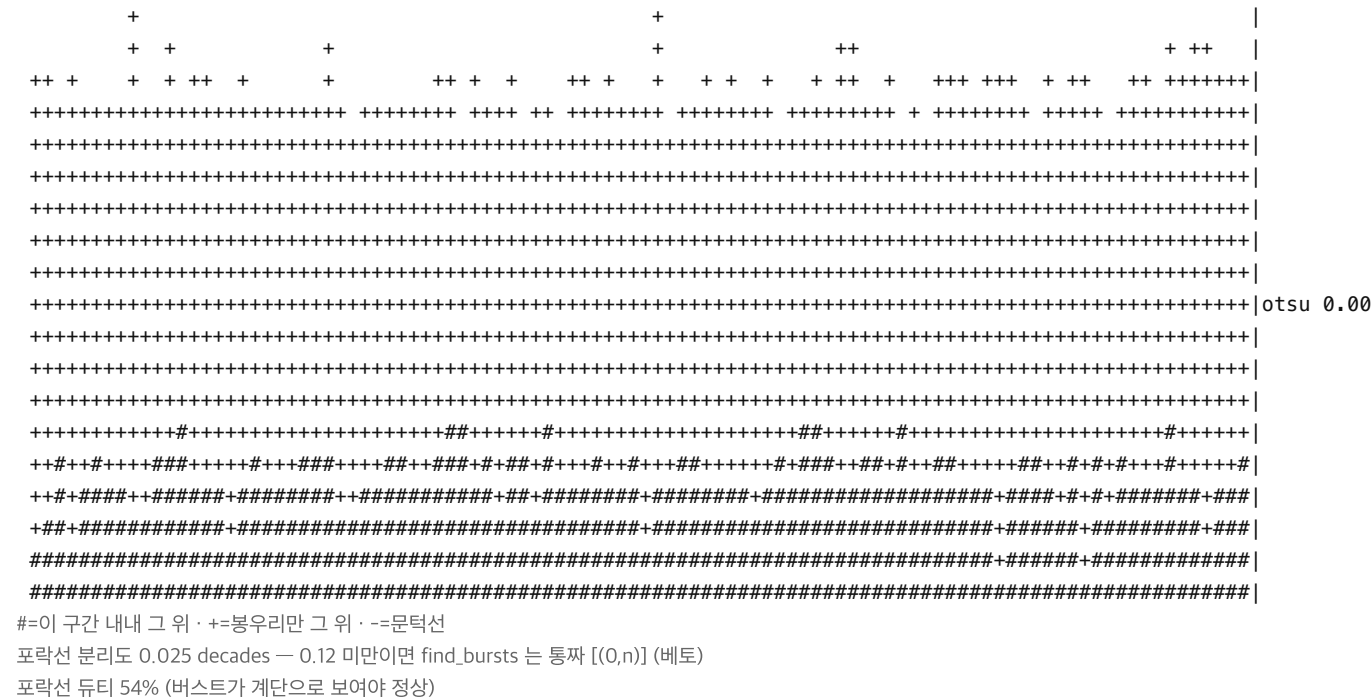
실캡처가 이렇게 보이면

a 줄 sp 가 30 을 넘고 c 줄 sk 가 r 과 비슷하면 이 경우다. 후보를 못 찾은 게 아니라 찾아 놓고 '임펄스가 만든 가짜' 로 판단해 버린 것이다 - 단계 6 의 피크/평균이 증거다.

꺼지지 않는 QPSK 한 개(-100 kHz, 심볼레이트 100 k)만 있다. 켜고 꺼지는 사건이 없으므로 레코드 전체가 곧 신호다.

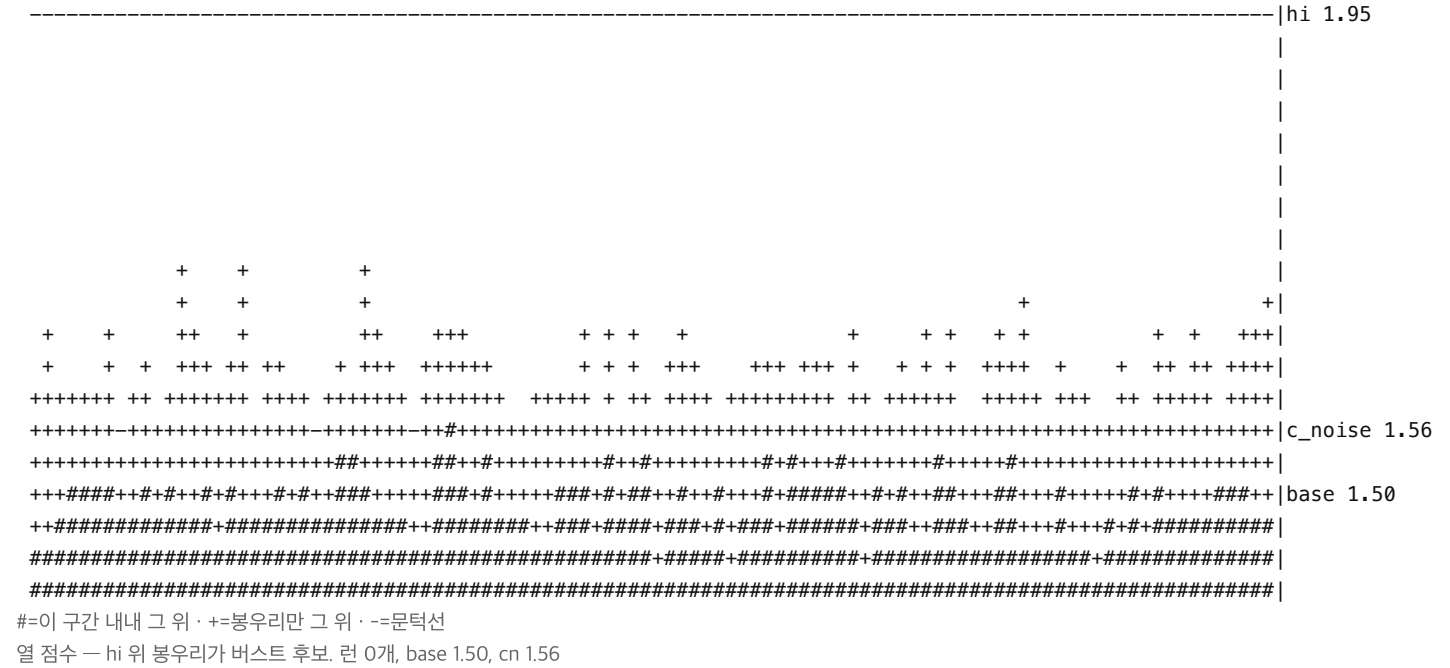
```
sigc a n65536 f1000000 ev2 ed54 sp5 dc0 cp2 cx1 #679h
sigc b c1023 g128 b150 cn156 t153 m171 dlo25 dhi45 #za94
sigc c r0 dn0 gp0 sb0 av0 ah0 sk0 #288g
sigc d kb20 kc19 w20 p-9 pd100 ps44 q999 qd0 qs0 #0vt9
```

단계 2 — 광대역 포락선 (여기서 막히면 E1 베토)

[illegible]

- : : : : = : : : * *, - # : . : - + = = , - , = : + , - - , . . . - : - + - , . . . : = : = : = , - , . . . - : : : = - : , + , . = # : - , . : - - - + , # - , . + - = - - : = :
= - + : = : - * : , - : + : - % = . . . - : : : = - + + + - - = . : , - = - : * + - - = - : * + : + + * - - - - * = * , = : - - = + : + + = - - , . : * = - + - : , # = - -
: # - = - , - : = = + + * - - : - , - = : . = = - - , - = + + = , - - . . . - = = = = : , + = : , - - + - + + = - : + - : * - : , . . . = , = - = : + - - : + - = - - , - : = :
: : : - - , . : + = + , : = : : , + , - - : . : + , - - , . : * , - - - , . : - , - - , - , + = = : : - = : * = : = : = + - , - + - : , = # . . - - : - : + , = + - , : = - -
: , . - - - , . : + - - = - - : - : , - - + - - = - : , + , - : , + , : : = , : + : : : , + = , - + * : = = - , : = = , : - - - : = : = - , = * - : - @ = : : ,
- : - - = : : . . : : = , : : : : . . - : , * : : - - . = = - , = - , - - + - - - : . : , : = , - - = + : * : = * - % : . : : + + = : = , : - : - - - , = - - + - = : , - , . : - - + + + ,
* = - - = = , = : + , . . . : - = , = * - - : : = = - * : : + : , = = , - * : : = * - : - = + + , = : , + , - - - * : : + , : : - : + : , - - - - : - : - , = : = , : - , - - : = :
= - * : : * : = * : : : , = - : - - - - : - - % # : . . : + - - + * - - - - , = - : - - - = + , = = - : * , . . . - - = = = : = , + , - : , = - : , . . - : : + - - # ,
* : , + , * : * - - * = = , + , + , - , - : + - # , . # = , - - + , . - = + = : , * : : - + * + - * : : : , - + : = - : : - , + % = , . + - = + : + - , = = : - , : - + *
빈별 바닥 대비 비율 — find_bursts 가 보는 그림. 세기가 일정한 연속 방사체는 여기서 사라지지만, 세기가
흔들리면 가짜 버스트로 남는다 (d 줄 pd 로 확인)

단계 5 — 열 점수와 문턱 (붕우리가 hi 를 넘어야 후보)



실캡처가 이렇게 보이면

c 줄 r0 이고 d 줄 pd100·kc 가 크면 이 경우다. 이때의 통짜는 오작동이 아니라 정답이다 — 이 캡처에서 찾을
버스트가 없다.